

## Programação Concorrente

Teste de Avaliação<sup>1</sup>

26 de maio de 2026

Duração: 2h00m

### I

- 1 Explique os diferentes motivos porque o `wait()` numa variável de condição de um monitor em linguagens modernas deve ser efectuado dentro de um ciclo que (re)testa um predicado.
- 2 Explique porque em Erlang, para esperar por uma mensagem de resposta a um pedido, não deve ser utilizado `receive X -> ...`, e diga como deve ser feito. Dê um exemplo, relativamente a uma operação que devolve uma resposta, descrevendo o protocolo e mostrando o código de uma função de interface.

### II

Considere uma abstração para controlo de acesso a  $R$  recursos. No máximo podem estar a ser acedidos  $S$  recursos simultaneamente, sendo  $S$  e  $R$  parâmetros da abstração, com  $S < R$ . Cada recurso pode ser acedido por várias threads concorrentemente. Pretende-se que escreva em Java, fazendo uso de primitivas baseadas em monitores, uma classe que implemente a interface:

```
interface Controller {  
    void request(int i) throws InterruptedException;  
    void release(int i);  
}
```

As operações têm como parâmetro o recurso ao qual uma thread pretende aceder (identificado por um número entre 0 e  $R - 1$ ); `request` deverá bloquear até o recurso poder ser acedido; `release` é invocado quando uma thread terminou o acesso ao recurso. Minimize as threads que são acordadas.

### III

Apresente o código Erlang para implementar o mesmo serviço descrito no grupo II, através de um ou mais processos. Suponha que os clientes são processos Erlang que comunicam pelo mecanismo nativo de mensagens, e implemente também as funções de interface apropriadas para serem usadas por estes para interagirem com o(s) processos relevante(s).